

Git Basics

Alexis Shakas Ian Wasser (TA)

ETH Zurich

2026-03-10

Lecture outline

Bookeeping	2
Version Control	6
Setup	13
Using Git	19
Git under the hood	35
Git in practice	40
Recap	48

Bookkeeping

Lecture	Date	T.H.A.	Room
1: Kickoff	17/2	N	RZ D8
2: AI assistance and tools	24/2	N	RZ D8
3: Project-based teamwork	3/3	N	RZ D8
4: Intro to Git	10/3	N	RZ D8
5: Git continued	17/3	N	ML H41.1
6: Git advanced	24/3	Y	RZ D8
7: AI-assisted Python	31/3	Y	ML H41.1
8: The 3 R's in Python	14/4	Y	ML H41.1
9: Python packaging ecosystem	21/4	N	RZ D8
10: OOP	28/4	Y	RZ D8
11: Testing and debugging	5/5	Y	ML H41.1
12: Joker lecture	12/5	N	RZ D8
13: Project distribution	19/5	N	RZ D8
14: Project marketplace	26/5	N	RZ D8

T.H.A.: Take home assignment



Important project dates

Keep these dates in mind for your project planning!

Milestone	Date	TODO
Project chosen	13/3	Discussion with me
Project Structured	27/3	Submit (markdown) file with project structure
Implementing	31/3	8 weeks to code and refine project
Project draft	15/5	Pre-submission of project for feedback
Project submission	29/5	Final submission of project on Git (tag)

Version Control

History of Git

Git is **free** and **open source** software started in 2005 from Linus Torvalds, to **collaborate** on the Linux Kernel.

Open Source refers to software that:

- Is freely available for anyone to **view, use, modify, and distribute**
- Encourages **collaboration, transparency, and community contribution**
- Often developed in public repositories (e.g., GitHub, GitLab)



You want to make changes to a project but you don't want to lose your old work

You want to make changes to a project but you don't want to lose your old work

- thesis.txt

You want to make changes to a project but you don't want to lose your old work

- thesis.txt
- thesis2.txt

You want to make changes to a project but you don't want to lose your old work

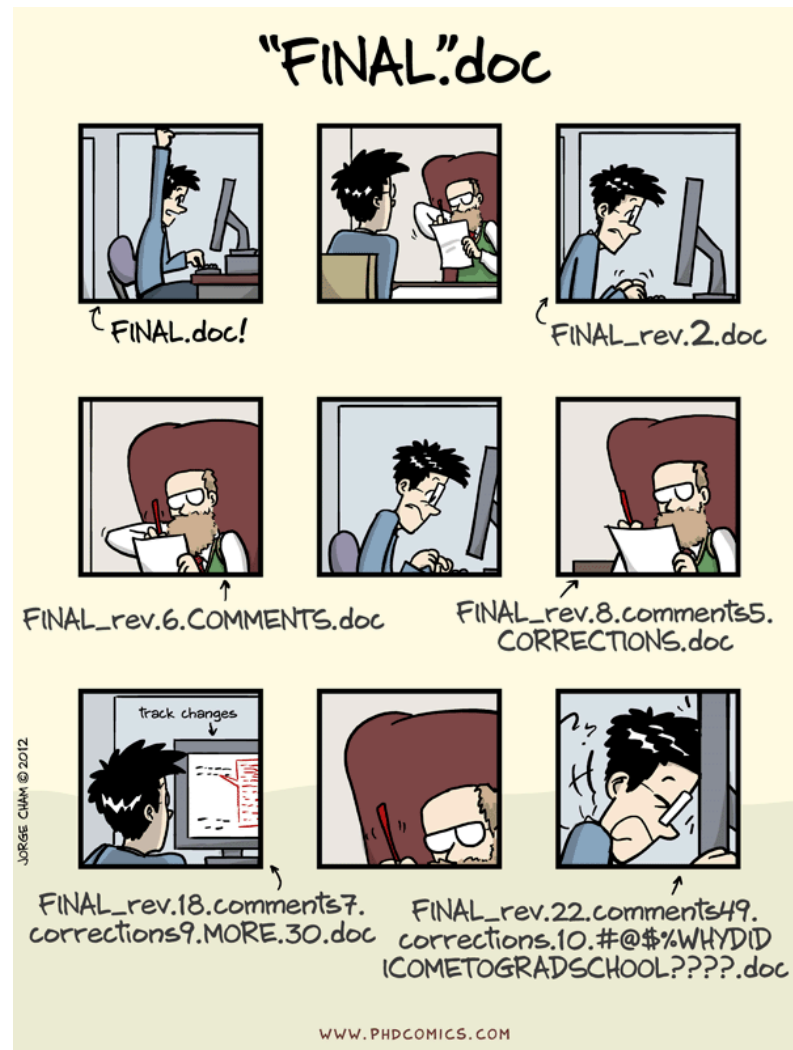
- thesis.txt
- thesis2.txt
- thesis2_final.txt

You want to make changes to a project but you don't want to lose your old work

- thesis.txt
- thesis2.txt
- thesis2_final.txt
- thesis2_final2.txt

You want to make changes to a project but you don't want to lose your old work

- thesis.txt
- thesis2.txt
- thesis2_final.txt
- thesis2_final2.txt
- ...



When saving to a file, its contents are usually **overwritten**.

Version control allows you to manage multiple versions of your project and **revert file contents** to previous versions.

When saving to a file, its contents are usually **overwritten**.

Version control allows you to manage multiple versions of your project and **revert file contents** to previous versions.

⇒ **Git** enables version control by storing **snapshots** of your files that you can revert to if needed.

Interacting with Git projects

You can interact with git in several ways!

You can interact with git in several ways!

- **Command Line Interface (CLI)** — the most powerful and flexible way.

Example commands: `git status`, `git commit`, `git push`

Interacting with Git projects

You can interact with git in several ways!

- **Command Line Interface (CLI)** — the most powerful and flexible way.

Example commands: `git status`, `git commit`, `git push`

- **Graphical User Interfaces (GUIs)** are useful for visualizing history, branches and more. Examples are GitHub Desktop, GitKraken and Visual Studio Code with the use of built-in Git tools or extensions.

You can interact with git in several ways!

- **Command Line Interface (CLI)** — the most powerful and flexible way.

Example commands: `git status`, `git commit`, `git push`

- **Graphical User Interfaces (GUIs)** are useful for visualizing history, branches and more. Examples are GitHub Desktop, GitKraken and Visual Studio Code with the use of built-in Git tools or extensions.
- **Web platforms** — for hosting, collaboration, and reviewing code. Popular examples are GitHub, GitLab and Bitbucket.

Git

Local to your
computer

Enables **Version
Control** of your
project



Git

Local to your
computer

Enables **Version
Control** of your
project



GitHub

Cloud Storage
for your Git
project

Hosted by
Microsoft



Git

Local to your
computer

Enables **Version
Control** of your
project



GitHub

Cloud Storage
for your Git
project

Hosted by
Microsoft



GitLab

**Open Source
Cloud Storage**
for your Git
project

Can be hosted by
you



Setup

You already have Git Installed

Now, we need to configure it

Configuration (Git Identity)

To set your Git identity, run the following commands in your terminal:

```
git config --global user.name "John Doe"  
git config --global user.email "john@doe.com"
```

These entries are stored with each “snapshot” to recognize who made what change. Note: The name or mail is **not** validated. You can enter any mail you want.

Git sometimes asks you for some user input and it opens a text editor for that. You can set your default editor for Git with the following command:

For example, to use **Visual Studio Code**:

```
git config --global core.editor "code --wait"
```

To use **Nano** (a minimal terminal editor):

```
git config --global core.editor "nano"
```


Here are two more useful configurations for Git that you can set right now:

```
git config --global init.defaultBranch "main"  
git config --global pull.rebase true
```

These:

- Set the default branch name to “**main**” (historically it was “master”)
- Tell Git to perform a rebase while pulling from a remote Git server.



We will have a more in depth look at this in 1 weeks

```
git config --global user.name "John Doe"  
git config --global user.email "john@doe.com"  
git config --global core.editor "code --wait" # or nano  
git config --global pull.rebase true
```

⇒ Configure it for yourself



Note:

The `--global` make the configuration effect every Git project. If you leave it away it only affects the current repository

Using Git

Repository: Your Git Project



Terminology

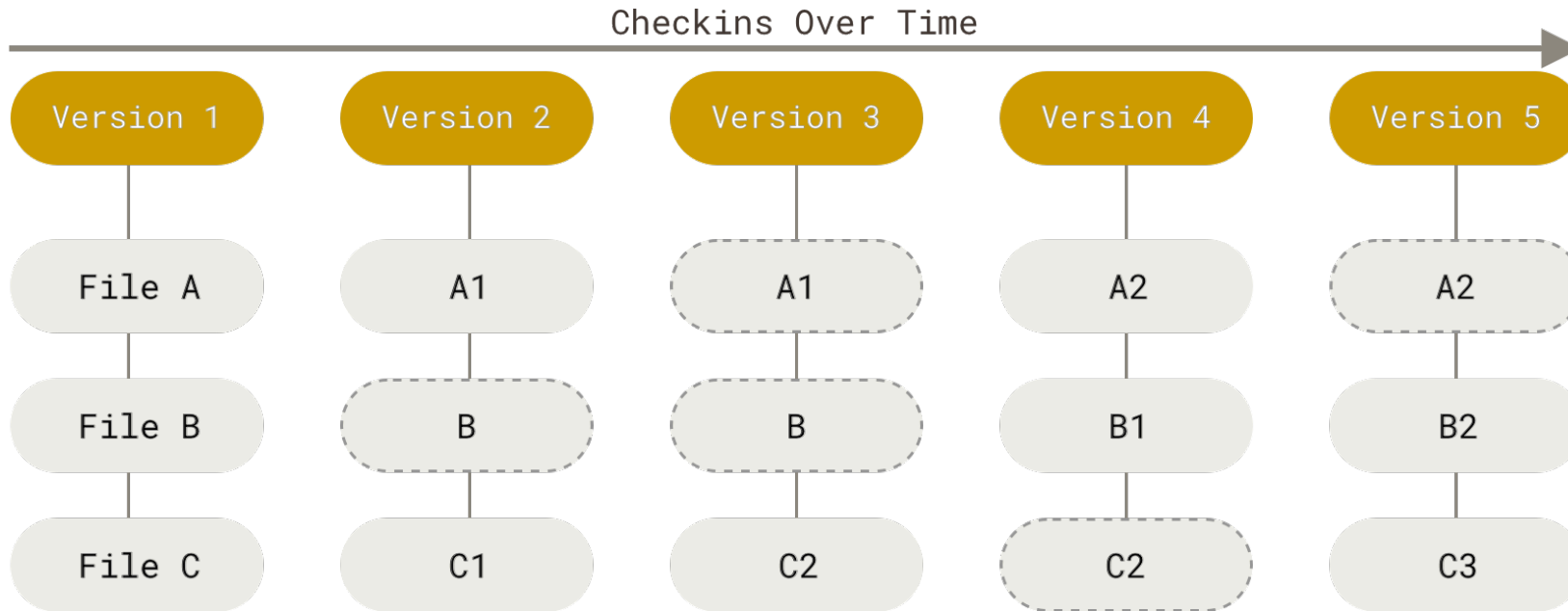
Repository: Your Git Project

Git History: The timeline of your project which contains all the **commits**; snapshots of your project at different points in time.

Terminology

Repository: Your Git Project

Git History: The timeline of your project which contains all the **commits**; snapshots of your project at different points in time.



Repository

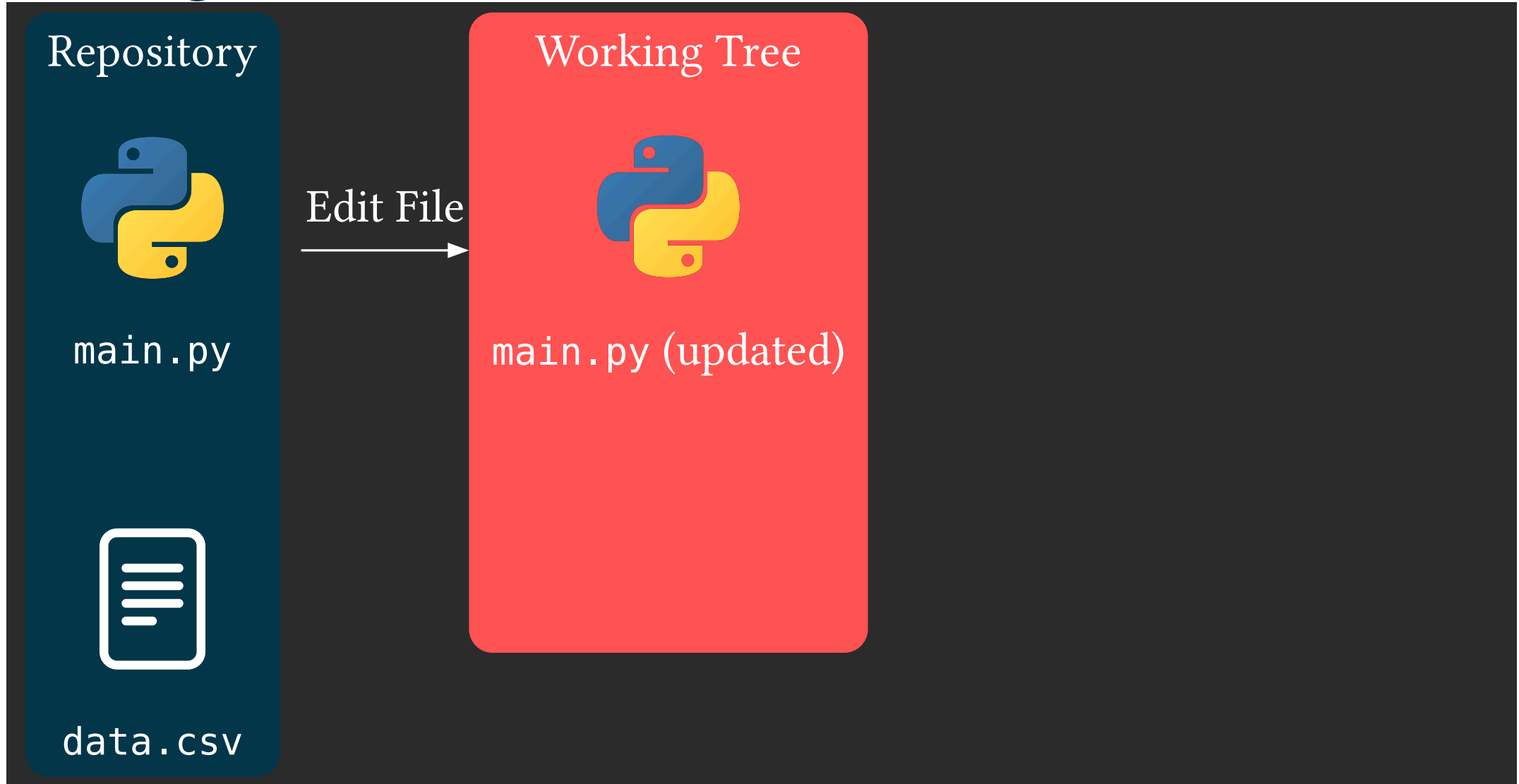


main.py

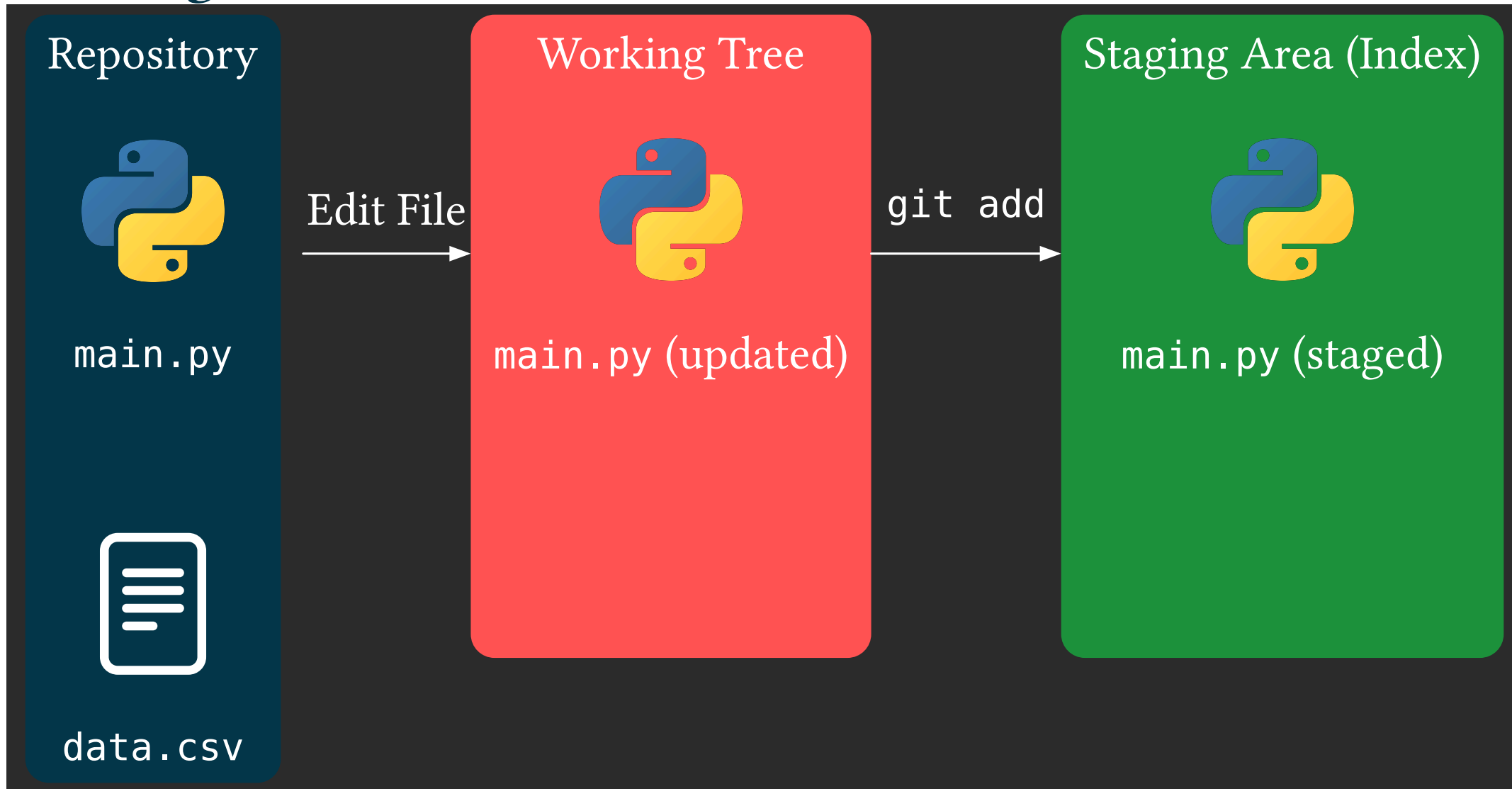


data.csv

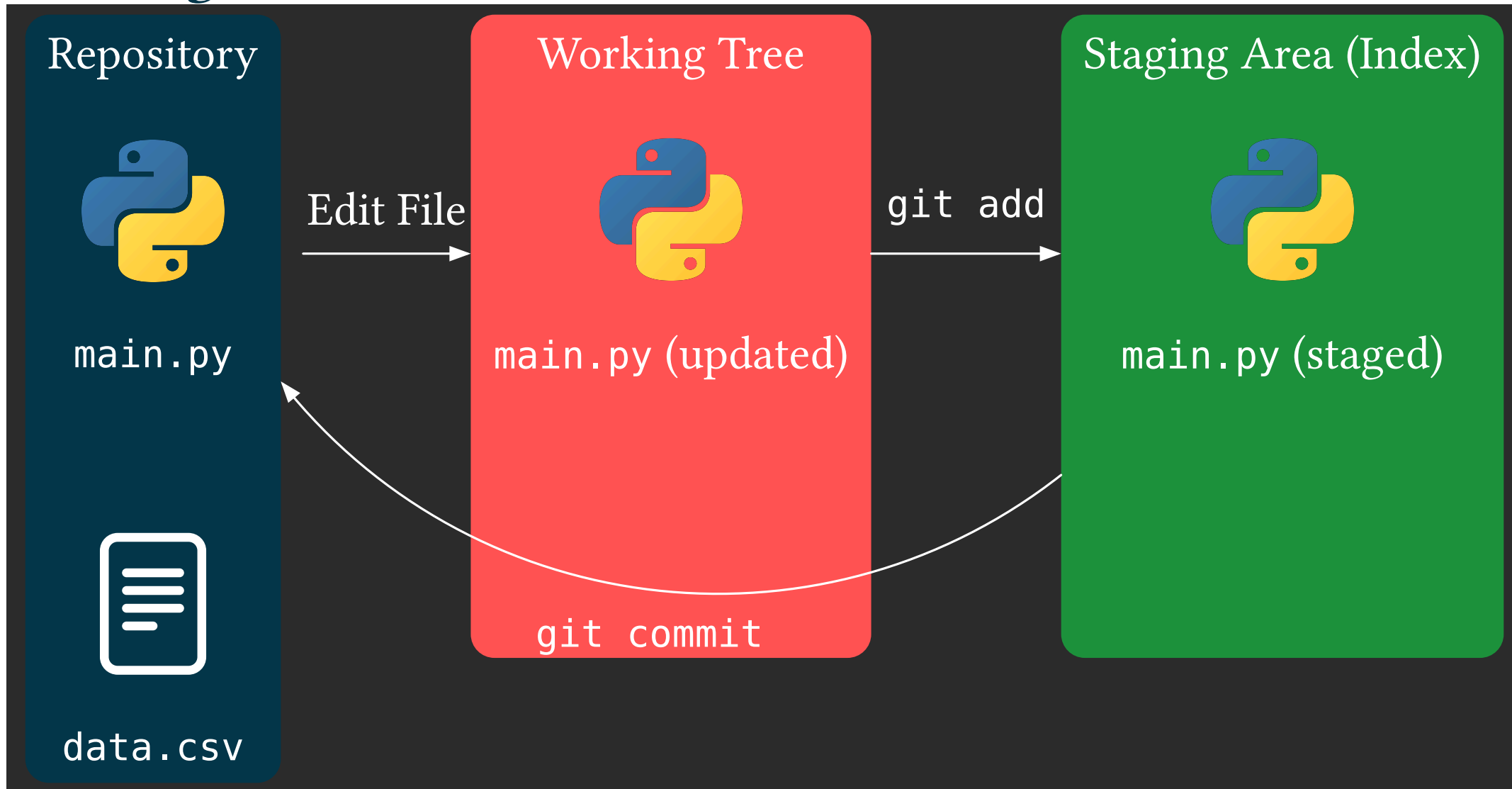
Creating a commit

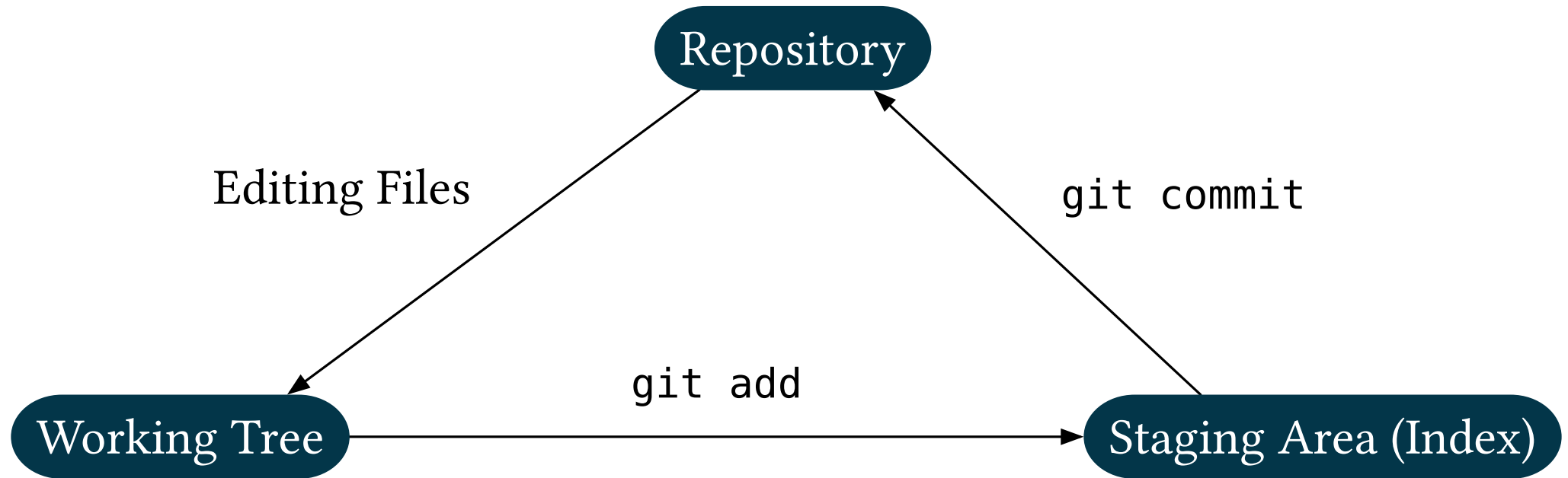


Creating a commit



Creating a commit





MiniTask: Create a Git Repository

We can create a Git Repository with the following commands

```
mkdir myFirstRepo # create a new (empty) directory  
cd myFirstRepo # go inside the directory  
  
git init # initialize the repository
```

The most important command in **Git**

```
git status
```



```
On branch main
```

```
No commits yet
```

```
nothing to commit (create/copy files and use "git add" to track)
```

Shows that the **Working Tree** and the **Staging Area** are empty

MiniTask: Create a commit

We can add files to the repository by creating a commit.

We create our first commit by adding a file to the directory

```
touch HelloWorld.txt
```

MiniTask: Create a commit

Using Git

We observe with `git status` that the **working directory** is **not clean** (not empty)

```
On branch main
```

```
No commits yet
```

```
Untracked files:
```

```
(use "git add <file>..." to include in what will be committed)
```

```
HelloWorld.txt
```

```
nothing added to commit but untracked files present (use "git add" to track)
```

MiniTask: Create a commit

We can then stage all files in our current directory by running

```
git add .
```


MiniTask: Create a commit

We observe with `git status` that we have now staged changes

On branch main

No commits yet

Changes to be committed:

(use "`git rm --cached <file>...`" to unstage)

new file: HelloWorld.txt

MiniTask: Create a commit

We can now finally commit the changes by running

```
git commit -m "My commit message"
```

MiniTask: Create a commit

We observe with `git status` that everything is clean again

```
On branch main  
nothing to commit, working tree clean
```

MiniTask: Observe the Commit

You can inspect the **Git History** by running

```
git log
```

MiniTask: Observe the Commit

You can inspect the **Git History** by running

```
git log
```



Note:

If the history is very long you might need to press “q” to quit the command

The output looks like this

```
commit f74c639917b73e7063b1f1fed8ff81aac31e73ac (HEAD ->
main)
```

```
Author: Ian Wasser <iwasser@ethz.ch>
```

```
Date:   Mon Feb 23 16:41:07 2026 +0100
```

```
My commit message
```

The output looks like this

```
commit f74c639917b73e7063b1f1fed8ff81aac31e73ac (HEAD ->
main)
Author: Ian Wasser <iwasser@ethz.ch>
Date:   Mon Feb 23 16:41:07 2026 +0100
```

My commit message

Here **f74c639917b73e7063b1f1fed8ff81aac31e73ac** is the commit hash. It **uniquely identifies** a commit.

MiniTask: Create a second commit

Try to create a commit by writing something into the file you just created

```
echo "Hello World!" > HelloWorld.txt
```

```
git add .
```

```
git commit -m "My second commit"
```


MiniTask: Create a second commit

You can observe the second commit also with `git log`.

```
commit dbbb91cd8f3777d738d1df2640f9cf514ac2ed96 (HEAD -> main)
```

```
Author: Ian Wasser <iwasser@ethz.ch>
```

```
Date:   Mon Feb 23 16:59:24 2026 +0100
```

My second commit

```
commit f74c639917b73e7063b1f1fed8ff81aac31e73ac
```

```
Author: Ian Wasser <iwasser@ethz.ch>
```

```
Date:   Mon Feb 23 16:41:07 2026 +0100
```

My commit message

Git under the hood

MiniTask: Inspect the changes of a commit

Git under the hood

We can inspect what has changed in a single commit by picking the first commit hash

e.g. **f74c639917b73e7063b1f1fed8ff81aac31e73ac**

and running

```
git diff f74c639917b73e7063b1f1fed8ff81aac31e73ac
```

This is show you the difference between the the latest and the specified commit

Git **doesn't save entire files**. Instead it saves the **differences** in the files.

Advantages:

- Fast
- Less disk space needed

Git Patch Files

Patch Files are files that contain the exact differences of certain versions of files.

We can create a patch file by running

```
git diff f74c639917b73e7063b1f1fed8ff81aac31e73ac >  
my_change.patch
```

Applying Patch Files

Another person could use this file and run

```
git apply my_change.patch
```

It then applies the same changes



Note:

This is important but for full collaboration it is not sufficient. We will take a look at collaboration in 2 weeks.

Git in practice

Sometimes we don't want Git to handle certain files

Examples:

- Password Files
- LLM API Keys
- .ds_store on Mac
- Build Files
- Cache Files

A special file called **.gitignore** makes sure that git ignores these files and won't react when does files change.

They **won't be added to the repository**

Create a gitignore file

To create a `.gitignore` file run

```
touch .gitignore
```



Note:

This file is just local to your repository

Create a gitignore file

You can then write into the file something like this

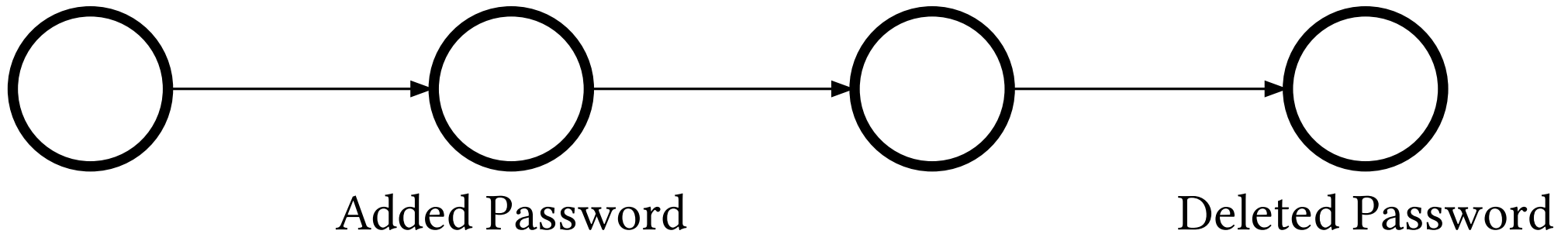
```
.ds_store/      # Excluding a full directory  
*.log           # Ignores all files that end with .log  
HelloWorld.txt  # Ignores this specific file
```

Some notes about gitignore files

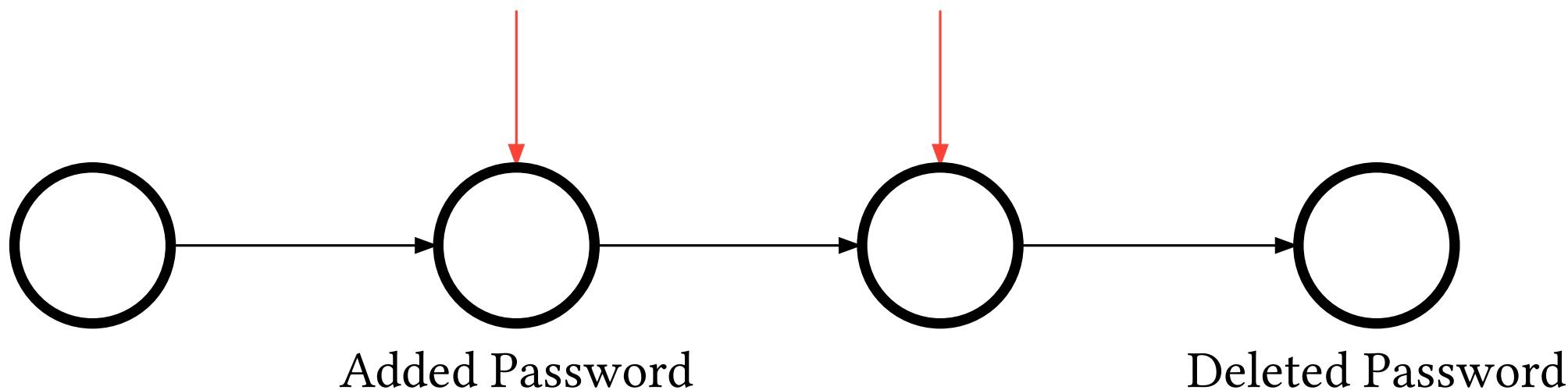
If the file you want to ignore has already been committed, the **.gitignore** file won't work.

Make sure that you add the **.gitignore** file first before committing

If you have **committed a file** you don't want to have in Git (like a password) and later **delete it** (and commit the deletion), the file is **still inside of the git history!!!**



If you have **committed a file** you don't want to have in Git (like a password) and later **delete it** (and commit the deletion), the file is **still inside of the git history!!!**



You can still observe the password with **git diff** in these commits

- Commit often and wisely
- Try to keep large binary files to a minimum (e.g. images, pdfs, ...) as this makes Git slow and repositories large
- Have short and descriptive commit messages (so if you come back later you know what happened)

Writing good commit messages promotes readability in your repository. There are several formats you can follow — just pick one and be consistent.

- Use a consistent format (e.g., Conventional Commits)
- Here is a cheatsheet
- Here some tips from from GitHub

MiniTask: Gitignore and file types

Download the list of files from Moodle and put them in your Git repository.

- 1) Add the files in your .gitignore file and observe the difference in `git status` before and after.
- 2) One by one, remove the files from the .gitignore file and observe how they appear again in `git status`
- 3) One by one, make changes to the files and commit the changes.
- 4) Use `git log` and `git diff` to inspect the history and changes.

Which of the files does Git work best with?

Recap

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.
- **Configure** Git with your identity, editor, and default settings.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.
- **Configure** Git with your identity, editor, and default settings.
- **Create** a Git repository and make commits using `git init`, `git add`, and `git commit`.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.
- **Configure** Git with your identity, editor, and default settings.
- **Create** a Git repository and make commits using `git init`, `git add`, and `git commit`.
- **Inspect** the Git history and diffs with `git log` and `git diff`.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.
- **Configure** Git with your identity, editor, and default settings.
- **Create** a Git repository and make commits using `git init`, `git add`, and `git commit`.
- **Inspect** the Git history and diffs with `git log` and `git diff`.
- **Understand** what a `.gitignore` file does and when to use it.

After this lecture, you should be able to:

- **Explain** what version control is and why it is useful.
- **Distinguish** between Git, GitHub, and GitLab.
- **Configure** Git with your identity, editor, and default settings.
- **Create** a Git repository and make commits using `git init`, `git add`, and `git commit`.
- **Inspect** the Git history and diffs with `git log` and `git diff`.
- **Understand** what a `.gitignore` file does and when to use it.
- **Avoid** common pitfalls for Git usage and know what file types Git works best with.